

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Engenharia de Software – LDAMD

PROJETO INTEGRADOR DA DISCIPLINA

Desenvolvimento de Sistema Distribuído com Aplicativo Móvel

Entrega Individual – 4 Sprints

Professores	Cleiton Silva Tavares e Cristiano de Macedo Neto
Disciplina	Lab. de Desenvolvimento de Aplicações Móveis e Distribuídas
Curso	Engenharia de Software
Período / Eixo	5º Período – Noite
Semestre Letivo	1º Semestre 2026
Modalidade	Individual
Tipo de Entrega	4 Sprints ao longo do semestre

1. Apresentação e Motivação

O Projeto Integrador é a atividade de maior peso na avaliação desta disciplina. Seu objetivo é permitir que o aluno aplique, de forma integrada e progressiva, os conhecimentos adquiridos ao longo do semestre: comunicação distribuída, middlewares orientados a mensagens (MOM), desenvolvimento de serviços web REST, desenvolvimento de aplicativos móveis com Flutter e, na etapa final, conceitos de computação em nuvem.

O projeto é de natureza individual e de livre escolha temática, desde que atenda aos requisitos arquiteturais definidos neste documento. A liberdade de escolha do domínio tem como objetivo aumentar o engajamento do aluno e estimular a criatividade na concepção de soluções reais.

A cada sprint o aluno entregará um incremento funcional e documentado do sistema, acumulando as entregas anteriores. O projeto foi repassado em 27/04/2026 e deve ser concluído até 03/07/2026, com quatro datas de entrega distribuídas ao longo desse período.

2. Requisitos do Projeto

2.1 Escopo Obrigatório

Todo projeto deve contemplar, obrigatoriamente, os seguintes elementos arquiteturais:

- **Aplicativo móvel para o cliente (usuário final):** desenvolvido em Flutter/Dart, permitindo que o usuário consuma o serviço oferecido pelo sistema (ex.: solicitar uma entrega, reservar um serviço, realizar um pedido).
- **Aplicativo móvel para o prestador de serviços:** também em Flutter/Dart, permitindo ao prestador receber notificações de novas demandas, aceitar ou recusar solicitações e atualizar o status das operações.
- **Backend em Web Service REST:** implementado em Flask (Python) ou Node.js (Express), expondo endpoints RESTful que intermediam a lógica de negócio e a persistência de dados.
- **Middleware Orientado a Mensagens (MOM):** a comunicação assíncrona entre os componentes deve utilizar um MOM (ex.: RabbitMQ, Redis Pub/Sub ou similar). O sistema deve ser orientado a eventos, com publicação e consumo de mensagens para eventos relevantes do domínio (ex.: nova solicitação criada, status atualizado, confirmação de aceite).

2.2 Arquitetura Orientada a Eventos

A adoção de uma arquitetura orientada a eventos (Event-Driven Architecture – EDA) é o princípio central do projeto. Isso significa que:

- As interações entre o backend e os aplicativos devem, preferencialmente, ocorrer por meio de eventos publicados em filas ou tópicos do MOM, e não apenas por chamadas REST síncronas.
- O aplicativo do prestador de serviços deve ser notificado de novas demandas de forma assíncrona, sem a necessidade de polling contínuo.
- Eventos de mudança de estado (ex.: pedido aceito, serviço concluído) devem ser propagados pelo MOM para os consumidores interessados.
- O aluno deve ser capaz de descrever o fluxo de eventos do seu sistema, identificando produtores, consumidores, tópicos/filas e o conteúdo das mensagens.

2.3 Exemplos de Domínios Aceitos

O aluno pode escolher qualquer domínio que satisfaça a estrutura cliente/prestador. Exemplos ilustrativos (não exaustivos):

- Plataforma de delivery (cliente faz pedido; entregador recebe e aceita a entrega)
- Marketplace de serviços domésticos (cliente solicita; profissional recebe e confirma)
- Sistema de agendamento de transporte (passageiro solicita corrida; motorista aceita)
- Plataforma de suporte técnico (usuário abre chamado; técnico recebe e atende)
- Sistema de reserva de serviços (cliente reserva; prestador confirma disponibilidade)

Domínios que não possuam uma distinção clara entre cliente e prestador de serviços não serão aceitos. O aluno deve submeter sua proposta de domínio até a data estipulada para a Sprint 1.

3. Entregas e Critérios de Avaliação

3.1 Visão Geral das Sprints

Sprint	Foco Principal	Entregas Esperadas	Pontos	Prazo
Sprint 1	Arquitetura e Backend REST	Proposta de domínio, diagrama de arquitetura, backend funcional com CRUD básico via REST, coleção de testes (Postman/Insomnia)	20	11/05/2026
Sprint 2	Integração com MOM	Configuração do MOM, publicação e consumo de eventos principais, demonstração de comunicação assíncrona entre backend e pelo menos	20	25/05/2026

Sprint	Foco Principal	Entregas Esperadas	Pontos	Prazo
		um consumidor, documentação dos eventos		
Sprint 3	Aplicativo Flutter (Cliente)	App Flutter funcional para o cliente, integração com backend REST e recebimento de notificações via MOM (ou polling assíncrono), telas de listagem, detalhes e ação principal	20	15/06/2026
Sprint 4	App Prestador + Integração Final	App Flutter para o prestador, fluxo completo cliente-MOM-prestador funcionando de ponta a ponta, demonstração gravada (screencast), relatório técnico final	20	03/07/2026

Observação: Esta disciplina não possui reavaliação (conforme Plano de Ensino). O aluno que não entregar uma sprint recebe zero naquela etapa, sem possibilidade de reposição.

3.2 Sprint 1 – Arquitetura e Backend REST

Objetivo

Definir o domínio do projeto, projetar a arquitetura do sistema e implementar o backend REST com as operações fundamentais de negócio.

Entregas

- Documento de Proposta (PDF, 1 a 2 páginas): descrição do domínio escolhido, justificativa, identificação dos dois perfis de usuário (cliente e prestador) e das principais funcionalidades.
- Diagrama de Arquitetura: representação visual dos componentes do sistema (apps, backend, MOM, banco de dados), com identificação dos protocolos de comunicação utilizados. Pode ser elaborado em Draw.io, Mermaid, C4 Model ou ferramenta equivalente.
- Backend REST funcional: mínimo de 4 endpoints implementados cobrindo as operações essenciais do domínio (ex.: criar solicitação, listar solicitações, atualizar status, consultar por ID).
- Banco de dados: utilização de SQLite (mínimo), PostgreSQL, MongoDB ou outro banco NoSQL. O schema deve ser documentado.
- Coleção de testes: arquivo Postman ou Insomnia exportado, com todos os endpoints documentados e com exemplos de requisição e resposta.

Critérios de Avaliação – Sprint 1 (20 pontos)

Critério	Peso	Pontuação Máx.
Clareza e viabilidade da proposta de domínio	20%	4,0
Qualidade e completude do diagrama de arquitetura	20%	4,0
Funcionalidade e correção dos endpoints REST	30%	6,0
Organização do código (Clean Architecture / boas práticas)	20%	4,0
Documentação dos endpoints (coleção de testes)	10%	2,0
TOTAL	100%	20,0

3.3 Sprint 2 – Integração com Middleware Orientado a Mensagens**Objetivo**

Integrar o MOM ao sistema, implementando a comunicação assíncrona orientada a eventos entre os componentes do backend.

Entregas

- MOM configurado e operacional: RabbitMQ, Redis Pub/Sub ou solução equivalente. Deve haver evidência de funcionamento (logs, screenshots ou vídeo curto).
- Produtor e consumidor implementados: o backend deve publicar eventos em pelo menos dois momentos distintos do fluxo de negócio (ex.: nova solicitação criada; status da solicitação alterado).
- Documentação dos eventos: tabela descrevendo cada evento (nome, produtor, consumidor, payload JSON de exemplo, tópico/fila utilizado).
- Demonstração de comunicação assíncrona: evidência de que um consumidor processa uma mensagem publicada pelo produtor sem que haja chamada REST direta entre eles.
- Relatório de integração (1 página): descrição das decisões de design relativas ao MOM (escolha da ferramenta, padrão utilizado, desafios encontrados).

Critérios de Avaliação – Sprint 2 (20 pontos)

Critério	Peso	Pontuação Máx.
MOM funcionando corretamente (evidência)	25%	5,0
Implementação de produtor e consumidor de eventos	30%	6,0
Qualidade e completude da documentação dos eventos	20%	4,0
Demonstração de assincronicidade real no fluxo	15%	3,0
Clareza do relatório de integração	10%	2,0
TOTAL	100%	20,0

3.4 Sprint 3 – Aplicativo Flutter para o Cliente

Objetivo

Desenvolver o aplicativo móvel Flutter destinado ao usuário cliente, com integração funcional ao backend REST e recebimento de atualizações de estado por meio do MOM ou mecanismo assíncrono equivalente.

Entregas

- App Flutter funcional para o cliente: mínimo de 3 telas (listagem de solicitações/serviços disponíveis, detalhes de um item, tela de criação/ação principal).
- Integração com o backend REST: o app deve consumir os endpoints implementados nas sprints anteriores.
- Atualização assíncrona de estado: o app deve refletir mudanças de estado do servidor sem exigir ação manual do usuário (ex.: quando o prestador aceitar uma solicitação, o app do cliente deve ser atualizado). Pode ser implementado via polling com intervalo definido, WebSockets ou integração com MOM.
- Arquitetura do app documentada: diagrama de camadas do app Flutter (models, services, widgets, screens), conforme padrão Clean Architecture discutido em aula.
- APK ou acesso ao código-fonte: o app deve ser executável. Entrega do código-fonte em repositório Git (GitHub, GitLab ou Bitbucket).

Critérios de Avaliação – Sprint 3 (20 pontos)

Critério	Peso	Pontuação Máx.
Funcionalidade do app (fluxo completo executável)	30%	6,0
Integração correta com o backend REST	25%	5,0
Atualização assíncrona de estado implementada	20%	4,0
Organização do código Flutter (Clean Architecture)	15%	3,0
Qualidade da interface (usabilidade e clareza)	10%	2,0
TOTAL	100%	20,0

3.5 Sprint 4 – Aplicativo do Prestador e Integração Final

Objetivo

Completar o sistema com o aplicativo Flutter para o prestador de serviços, garantindo o funcionamento do fluxo completo de ponta a ponta, desde a solicitação do cliente até a conclusão pelo prestador, com comunicação assíncrona via MOM.

Entregas

- App Flutter funcional para o prestador: mínimo de 3 telas (lista de solicitações pendentes, detalhes da solicitação com opção de aceitar/recusar, acompanhamento das solicitações em andamento).

- Notificação assíncrona ao prestador: quando o cliente criar uma solicitação, o app do prestador deve ser notificado (via MOM, WebSocket ou polling), sem que o prestador precise atualizar manualmente a tela.
- Fluxo completo funcionando: demonstração de que o sistema funciona de ponta a ponta: cliente cria solicitação → backend publica evento no MOM → prestador é notificado → prestador aceita → cliente é notificado da atualização.
- Screencast de demonstração: vídeo de 3 a 5 minutos demonstrando o sistema em operação, com os dois apps rodando simultaneamente (pode ser em emuladores). O vídeo deve cobrir o fluxo completo descrito acima.
- Relatório Técnico Final (mínimo 4 páginas): descrição da arquitetura implementada, decisões de design, dificuldades encontradas e soluções adotadas, reflexão sobre os padrões estudados (EDA, MOM, Clean Architecture, REST). O relatório deve incluir pelo menos 3 referências bibliográficas da ementa da disciplina ou de fontes acadêmicas indexadas.
- Repositório Git organizado: código-fonte de todos os componentes (backend, app cliente, app prestador), com README descritivo e instruções de execução.

Critérios de Avaliação – Sprint 4 (20 pontos)

Critério	Peso	Pontuação Máx.
Funcionalidade do app do prestador	25%	5,0
Fluxo completo de ponta a ponta funcionando	30%	6,0
Notificação assíncrona ao prestador via MOM/evento	20%	4,0
Qualidade e clareza do screencast	10%	2,0
Relatório técnico final (profundidade e referências)	15%	3,0
TOTAL	100%	20,0

4. Calendário de Entregas

O projeto foi repassado em 27/04/2026. A entrega final deve ocorrer até a primeira semana de julho de 2026, respeitando o fechamento do diário acadêmico (03/07/2026). As datas abaixo são definitivas:

Sprint	Foco	Data de Entrega
Sprint 1	Proposta + Backend REST	11/05/2026
Sprint 2	Integração MOM	25/05/2026
Sprint 3	App Flutter – Cliente	15/06/2026
Sprint 4	App Flutter – Prestador + Entrega Final	03/07/2026

Atenção: o término das aulas para veteranos (18 semanas) está previsto para 30/06/2026 e o fechamento do diário para 03/07/2026. Entregas realizadas após 03/07/2026 não serão aceitas, independentemente do motivo.

5. Regras e Orientações Gerais

5.1 Integridade Acadêmica

O projeto é estritamente individual. É vedado:

- Compartilhar código-fonte entre colegas, mesmo que parcialmente.
- Submeter código gerado integralmente por ferramentas de IA generativa sem compreensão e adaptação pelo aluno. O uso de assistência de IA para fins de auxílio à aprendizagem é permitido, desde que o aluno consiga explicar e defender todas as escolhas de implementação.
- Plagiar soluções de projetos de semestres anteriores ou de repositórios públicos sem a devida adaptação e referência.

Casos de plágio ou compartilhamento de código serão tratados conforme o Regimento Acadêmico da PUC Minas, podendo resultar em nota zero para todos os envolvidos.

5.2 Tecnologias

As tecnologias obrigatórias são: Flutter/Dart para os aplicativos móveis; Flask (Python) ou Express (Node.js) para o backend; SQLite ou PostgreSQL para persistência; RabbitMQ, Redis Pub/Sub ou solução equivalente documentada para o MOM.

O uso de tecnologias adicionais (ex.: Docker para containerização do MOM, Firebase para notificações push, Cloud para implementação) é encorajado, mas não obrigatório. O aluno deve justificar suas escolhas tecnológicas no relatório final.

5.3 Repositório Git

É obrigatório o uso de Git desde a Sprint 1. O repositório deve ser público (ou compartilhado com o professor) e conter histórico de commits representativo do desenvolvimento. Repositórios com apenas um commit na entrega final serão penalizados em 20% da nota da sprint correspondente.

5.4 Dúvidas e Suporte

O professor estará disponível durante os encontros presenciais e pelo canal oficial da disciplina no AVA para esclarecimento de dúvidas. Questões técnicas devem ser postadas no fórum da disciplina para que todos os alunos se beneficiem das respostas.

6. Referências Bibliográficas

As referências a seguir fundamentam as escolhas arquiteturais e tecnológicas deste projeto e devem ser consultadas pelo aluno ao longo do desenvolvimento:

MARTIN, Robert C. Arquitetura limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019. (Fundamenta os princípios de Clean Architecture adotados na organização dos apps Flutter e do backend.)

BAILEY, Thomas. Flutter for beginners. 3rd ed. Birmingham: Packt, 2023. (Referência principal para o desenvolvimento dos aplicativos móveis com Flutter 3.10+ e Dart 3.x.)

HOHPE, Gregor; WOOLF, Bobby. Enterprise Integration Patterns: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003. (Padrões de integração por mensagens: filas, tópicos, pub/sub, pipes and filters. Base teórica para o MOM.)

RICHARDSON, Chris. Microservices patterns: with examples in Java. Shelter Island: Manning, 2018. (Padrões de EDA, Saga, e comunicação assíncrona entre serviços.)

COULOURIS, George et al. Distributed Systems: concepts and design. 5th ed. Boston: Addison-Wesley, 2011. (Conceitos de sistemas distribuídos, comunicação indireta e middlewares.)

JOURNAL OF CLOUD COMPUTING. London: Springer, 2012-. ISSN 2192-113X. Disponível em: <https://journalofcloudcomputing.springeropen.com/>. (Periódico para consulta em relação aos aspectos de nuvem da Sprint 4.)

Obs.: Para o Relatório Técnico Final (Sprint 4), o aluno deve citar no mínimo 3 referências, preferencialmente das listadas acima ou de artigos indexados no IEEE Xplore, ACM Digital Library ou periódicos CAPES.